Programmation procédurale

Labo 3 - Codebreaker en C

Partie 2 - Architecture multifichier et premiers pointeurs

Cotation	20 points - entre dans les 30 % « Laboratoires et travaux »
Échéance	21 octobre 2026 - 17 h 30
Remise	GitHub personnel / labo3 + fichier .zip sur Classroom

Remise obligatoire

Le projet doit être présent dans le dossier labo3 de votre GitHub personnel ET remis au format .zip sur Classroom avant l'échéance. Si l'une des deux remises manque ou est hors délai : 0/20.

But du laboratoire

Développer en C un jeu de décryptage de code secret en console. Ce travail prolonge les rappels de programmation procédurale et impose désormais une organisation en plusieurs fichiers ainsi que l'utilisation de pointeurs simples pour modifier des variables depuis une fonction.

Règles imposées

- Le code secret contient de 4 à 6 chiffres selon le mode ; chaque chiffre est compris entre 0 et 9.
- Tous les chiffres d'un même code sont distincts, y compris ceux d'une proposition.
- Après chaque tentative valide, afficher le nombre de chiffres bien placés et mal placés.
- Un chiffre mal placé est présent dans le code secret, mais à une autre position.
- Une proposition invalide est refusée et ne consomme aucune tentative.
- Le programme se termine proprement après le choix de quitter ; une nouvelle partie est possible sans relancer l'application.

Menu principal

Au lancement, l'utilisateur peut choisir entre une partie classique, une partie personnalisée, un mode deux joueurs ou quitter. La présentation et la numérotation du menu sont libres.

Fonctionnement

1. Partie classique

- Le programme génère aléatoirement un code de 4 chiffres distincts.
- Le joueur dispose de 10 tentatives valides pour découvrir le code.
- Après chaque proposition, afficher les deux indices : bien placés et mal placés.
- Victoire si les 4 chiffres sont bien placés ; défaite après la dernière tentative.

2. Partie personnalisée

- Le joueur choisit une longueur de code comprise entre 4 et 6 chiffres.
- Le joueur choisit entre 5 et 15 tentatives.
- Le programme génère un code adapté à ces paramètres.
- Les mêmes règles de comparaison et de victoire s’appliquent.

3. Mode deux joueurs

- Le joueur A saisit un code secret de 4 chiffres distincts.
- Le joueur B tente de le découvrir en 10 tentatives.
- Le code secret ne doit pas être affiché pendant les propositions du joueur B.
- Le programme donne les mêmes indices qu’en mode classique.

Historique et fin de partie

- Conserver les propositions valides et leurs deux indices dans un tableau à deux dimensions ou dans plusieurs tableaux.
- Permettre au joueur de consulter l’historique de la partie en cours.
- À la fin, afficher victoire ou défaite, le nombre de tentatives utilisées et le code secret.
- Calculer un score à partir des tentatives restantes ; en mode personnalisé, tenir également compte de la longueur du code. La formule choisie doit être expliquée dans le README.

Exemple de déroulement

```
Code de 4 chiffres - 10 tentatives
Proposition : 1234
Bien placés : 2 | Mal placés : 1
Proposition : 1532
Bien placés : 4 | Mal placés : 0
Victoire en 2 tentatives !
```

Exemple fictif d’affichage : les indices dépendent du code secret réellement choisi.

Contraintes techniques

Architecture multifichier

Séparer le menu, la logique du jeu et la génération / comparaison des codes. Organisation minimale attendue :

```
labo3/
  main.c
  code.h / code.c
  jeu.h / jeu.c
  Makefile
  README.md
```

- Les fichiers .h déclarent les fonctions et les types partagés ; ils possèdent des gardes d'inclusion.
- Les fichiers .c contiennent les implémentations ; main.c reste consacré au lancement et au menu.
- Le Makefile doit permettre de construire le programme avec make et de supprimer les fichiers générés avec make clean.

Pointeurs simples

Utiliser des paramètres pointeurs pour permettre à une fonction de modifier au moins deux résultats calculés. Par exemple, la comparaison de deux codes peut mettre à jour séparément le nombre de chiffres bien placés et mal placés.

- Manipuler les opérateurs & et * pour transmettre et modifier des valeurs.
- Savoir expliquer le passage par valeur et la modification d'une variable via son adresse.
- Aucune allocation dynamique, liste chaînée ou pointeur de fonction n'est demandé.

Qualité et robustesse

- Refuser les chiffres répétés, les longueurs incorrectes et les entrées non numériques.
- Éviter de dupliquer la comparaison des codes entre les trois modes.
- Découper les traitements en fonctions simples et compréhensibles.
- Prévoir un README indiquant comment construire et lancer le programme.

Évaluation

Cotation - 20 points. Une solution partielle peut obtenir des points si les fonctionnalités réalisées sont correctes et si le code est structuré et compris.

Critère	Points
Mode classique et comparaison des codes	4
Mode personnalisé et validations	3
Mode deux joueurs	2
Historique, score et fin de partie	3
Architecture .c / .h et Makefile	3
Utilisation correcte des pointeurs simples	3
Lisibilité, fonctions, robustesse et README	2
TOTAL	20

Qualité attendue

- Ne placez pas toute la logique dans main().
- Une fonction doit avoir une responsabilité clairement identifiable.
- Réutilisez les traitements communs aux trois modes.
- Commentez les décisions techniques importantes plutôt que chaque instruction.
- Soyez capables d’expliquer les paramètres pointeurs, les résultats et les choix algorithmiques.

Important

Le laboratoire évalue la compréhension et non la seule production d’un programme fonctionnel. Une vérification orale ponctuelle peut porter sur une fonction, une modification demandée en direct ou l’explication d’un choix technique.

L’usage d’outils d’assistance est autorisé pour rechercher, comprendre et déboguer. Le code remis doit néanmoins être personnellement compris et défendable.